Project Brief: Vigil, a sensor monitor that fails loudly

Built for: Schneider Electric Summer 2027 Software Engineer Intern, NetBotz team, Andover MA (iCIMS 2026-131307) **Version 3, Sept 25, 2026.** Replaces versions 1 and 2. Scope narrowed after review; several technical claims in version 2 were wrong or overstated and are corrected here (see Section 9).

---

1. The promise, in one sentence

Vigil detects stale or invalid sensor data, and keeps monitoring predictably when its database or its data sources fail.

Everything in the project exists to make that sentence true and to prove it. The goal is credible evidence that you can do this team's intern work (test carefully, fix defects, treat failure modes seriously, work in C on Linux), not a replica of every technology in the NetBotz product.

The interview demonstration: stop a sensor, stop the database, restore both, then show exactly what was detected, what was preserved, what was lost, and what recovered, with counts and timestamps.

---

2. What you build

- **A pod simulator** (Python, python-can) that sends readings from one or two simulated sensor pods over a virtual CAN bus (`vcan0`, Linux SocketCAN). Start with two channels only: temperature and a door state. Each pod also sends a heartbeat.
- **vigild**, a C daemon on Linux that reads the bus, validates every frame, tracks whether each data source is alive and making progress, raises alarms with hysteresis, and records history in PostgreSQL.
- **A test suite** that reproduces each failure on demand and checks the daemon's behavior against a written expectation.

Why CAN: NetBotz connects its sensor pods over A-Link, which Schneider's user guide describes as a proprietary CAN bus, and Linux can exchange CAN frames on a virtual interface with no hardware. You design your own small protocol on the same kind of bus. You are not reproducing A-Link, and you should say so.

---

3. Design

3.1 The pod protocol

Classic CAN frames (11-bit ID, up to 8 data bytes).

CAN ID	Message
0x100 + pod ID	Reading: channel, value (int16; 0.01 °C, or 0/1 for the door), status, rolling counter, CRC-8
0x700 + pod ID	Heartbeat: pod state and uptime (0x700 + node ID is the CANopen convention for heartbeats)

What the counter and CRC can and cannot do:

- **They detect:** corrupted payloads, a pod repeating the same frame (counter not advancing), lost frames (counter jumps), and two sources emitting interleaved sequences on one ID.
- **They cannot establish who sent a frame.** Anyone on the bus can compute a valid CRC and continue a counter. Treat sequence problems as anomalies worth an alarm, not as proof of spoofing or of a duplicate pod.

3.2 Three separate questions per data source

Version 2 lumped these together. They are different failures and get different handling:

Question	Signal	What it means when it fails
Freshness: is data arriving?	Time since the last valid reading; heartbeat timeout	Pod or link is down. Raise "communication lost." The last value is shown as stale, never as current.
Progress: is the publisher alive?	Rolling counter advancing	A repeated counter means a stuck or looping publisher. Repeated frames are rejected and counted.
Plausibility: does the measurement make physical sense?	Range limits; maximum rate of change; unexplained flat readings	Out-of-range or impossible jumps mark the value invalid. A flat reading is only **suspicious**: a stable room can produce one. Flag it after a configurable window; never call it a proven failure.

3.3 The daemon's structure

- **Main thread:** one `poll()` loop over the CAN socket, a `timerfd` for timeouts and housekeeping, and a `signalfd` for clean shutdown. It decodes frames, runs the three checks, and updates alarm state. **It never calls the database.**
- **Persistence thread:** owns the libpq connection and makes blocking calls with `connect_timeout` and a server `statement_timeout`. Version 2 put database writes in the poll loop, where one slow or hung call would have stalled sensor processing and timeout checks. A ring buffer alone doesn't fix that; a separate thread does.
- **Between them:** a bounded queue protected by a mutex. Rule: the writer copies a batch out while holding the lock, releases it, and only then does I/O, so the main thread never waits on the network.
- **Alarm state lives in the daemon's memory** and is always current. The database is history. If history is lost during an outage, the writer records a gap marker and writes the current alarm state when the connection returns.
- **Alarm engine:** a threshold with hysteresis (for example, raise above 27 °C, clear below 26 °C), severities, and an active/cleared lifecycle.
- **Counters for everything:** frames received, frames rejected by reason, readings queued, written, and dropped, and outage duration. These are what make the demonstration exact.
- **systemd service** with `Type=notify` and `WatchdogSec`, running as an unprivileged user.

3.4 Durability limits, stated up front

- During a database outage, readings are preserved **only up to the queue's capacity and only while the daemon keeps running.**
- When the queue fills, the oldest readings are dropped, the drops are counted, and a "persistence degraded" alarm is raised.
- **If the daemon crashes, whatever is still in the queue is lost.** Surviving that would take an on-disk spool, which is a possible later extension with its own costs (disk wear, write latency, recovery logic).

---

4. Evidence

Each test states its expected behavior in writing before it runs.

Scenario	Expected result
Pod goes silent	"Communication lost" within the timeout; value marked stale

Malformed frame (wrong length, bad CRC)	Rejected and counted by reason; no alarm change
Duplicate or replayed frame	Rejected as non-advancing; counted
Counter jump	Loss counted; data still accepted
Value oscillating around the threshold	One alarm, not a flapping series (hysteresis)
Database stopped, then restored	Monitoring and alarms continue; queued readings backfill; gap recorded if any were lost
Outage longer than queue capacity	Exact drop count matches the configured capacity; "persistence degraded" raised
Daemon killed during an outage	Documented loss happens exactly as stated in 3.4; systemd restarts the daemon
High message rate	The daemon keeps up or reports what it dropped. This measures **application behavior under load only.** `vcan` has no arbitration, bitrate limit, or electrical behavior, so it says nothing about a physical bus.

Plus:

- **C unit tests** (cmocka) for the decoder, CRC, counter logic, hysteresis, and the queue.
- **One libFuzzer target** on the frame decoder with AddressSanitizer and UndefinedBehaviorSanitizer. It's small, and it's the systematic version of the malformed-frame tests.
- **A manual test checklist** run with `candump` and `cansend`, recording expected vs. actual.
- **CI** on GitHub Actions for build, unit tests, and a short fuzz run. Loading the `vcan` module on hosted runners isn't guaranteed; if it fails, keep the bus integration tests as a local gate and say so in the README.

---

5. Order of work

1. **Working core:** Ubuntu ARM64 VM (Docker Desktop can't load `vcan`), `vcan0`, simulator, daemon with freshness/progress/plausibility checks, alarms with hysteresis, persistence thread, PostgreSQL history.
2. **Evidence:** the scenario table above, unit tests, the fuzz target, the manual checklist, and a scripted version of the interview demonstration.
3. **One extension, chosen after the core works.** My default is a **Buildroot image**: "Embedded Linux" leads their stack, and it's rare for a freshman to have built one. A small Java HTTP API adds less, because the posting accepts Java, JavaScript, or C and you already have strong JavaScript. If the core runs long, skip the extension; a finished core with evidence beats two half-built layers.

Deferred (separate projects later, if ever): a handwritten Modbus TCP server, signed configuration backups, a full dashboard, authenticated CAN frames, real hardware.

If you build the Buildroot extension

- The daemon inside a QEMU guest does **not** see `vcan0` in the outer Ubuntu VM. Each kernel has its own. Run the simulator inside the guest at first: replaying recorded scenarios with `canplayer` from can-utils avoids putting Python in the image.
- Connecting the guest to the outer VM's bus later is possible with QEMU's CAN emulation (an emulated CAN controller in the guest, bridged to host SocketCAN), but it's optional.
- Buildroot runs only on Linux; build inside the Ubuntu VM.

---

6. Hardware later: what carries over and what doesn't

- **Carries over:** the daemon's CAN code (bind to `can0` instead of `vcan0`), its logic, the database schema, and most tests.
- **Is new work:** a board-specific image (a QEMU image is not a Raspberry Pi image), the CAN controller driver and device-tree setup, bitrate configuration, termination and wiring, and validation on real hardware, including everything `vcan` can't simulate.

---

7. Build it so you can defend it

Write the decoder, the three checks, the alarm state machine, and the queue handoff yourself before using Claude Code to review and extend them. Be able to answer:

- Step by step, what happens when a pod goes silent? When the database goes down? When both come back?

- How do you tell a dead pod from a quiet one, and a stuck sensor from a stable room?
- Why a persistence thread? What do the threads share, and how is it protected?
- What's lost if the daemon crashes mid-outage, and what would fixing that cost?
- What can your counter and CRC detect, and what can't they?
- What can a `vcan` load test tell you, and what can't it?
- What did the fuzzer find?

---

8. Resume bullets for later (fill in real numbers; don't use until built)

Vigil: Sensor Monitoring Daemon | C, Linux (SocketCAN, systemd), PostgreSQL, Python

- Built a C daemon on Linux that monitors simulated sensor pods over SocketCAN, tracking data freshness, publisher progress, and measurement plausibility separately so stale or invalid data raises an alarm instead of looking healthy
- Isolated PostgreSQL writes in a persistence thread behind a bounded queue so monitoring continues through database outages; tested outage, overflow, and recovery with exact counts of preserved and dropped readings
- Wrote [N] reproducible fault tests (silent pods, malformed, replayed, and out-of-sequence frames, threshold oscillation, database loss) plus a libFuzzer harness for the frame decoder
- If built: packaged the daemon and PostgreSQL into a custom ARM64 Embedded Linux image with Buildroot, booted under QEMU

On the Schneider resume, Vigil replaces **Attest** and the "Additional Projects" line. KirtanCut stays, because its outside adoption is evidence nothing else on the page provides.

---

9. Corrections to version 2

Version 2 said	Correct statement
A high-priority flood tests CAN bus overload	On `vcan` it only tests the application under a high message rate
Conflicting counters reveal spoofed or duplicate pods	They reveal sequence anomalies; they don't establish sender identity
An unchanged value means a stuck sensor	Unexplained flat readings are suspicious, not proof of failure

One poll() loop does the database writes	Blocking libpq calls would stall monitoring; writes move to a persistence thread
Readings survive a database restart	Only within queue capacity, and only while the daemon is running
Switching `vcan0` to `can0` keeps everything, including the image	The daemon's CAN code carries over; the image, drivers, bus setup, and validation don't
(missing)	A daemon in a QEMU guest can't see the outer VM's `vcan0`
The "four-week" plan across ten technologies	Too broad for a freshman quarter; narrowed to a core, evidence, and at most one extension

---

10. Timing

Submit the Schneider application as soon as the resume's open claims are resolved; that's days, not weeks. Build Vigil for the interview stage. Uploading a newer resume to your candidate profile later may not reach whoever already screened the application, so the dependable routes are linking the repo in your next applications and bringing it up directly with any recruiter who contacts you.

Sources

- Schneider Electric posting, ID 2026-131307: https://careers-se.icims.com/jobs/131307/software-engineer-intern/job
- NetBotz 5.x User Guide (A-Link described as a proprietary CAN bus): https://radiotrade.s3.amazonaws.com/APC%20Resources/NetBotz%205%20User%20Guide.pdf
- Linux SocketCAN documentation (virtual CAN interfaces): https://docs.kernel.org/networking/can.html
- PostgreSQL libpq documentation (blocking vs. asynchronous command processing): https://www.postgresql.org/docs/current/libpq-async.html
- QEMU CAN bus emulation: https://www.qemu.org/docs/master/system/devices/can.html
- vcan in a Colima/Lima Ubuntu VM on macOS: https://github.com/abiosoft/colima/discussions/1108
- Docker Desktop for Mac and SocketCAN: https://github.com/docker/for-mac/issues/4405
- Schneider security notification SEVD-2026-223-02 (NetBotz 5 750/755), via CISA ICSA-26-260-05: https://www.cisa.gov/news-events/ics-advisories/icsa-26-260-05